



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 8: Neuroevolution and Deep Optimisation for Intelligent Systems

Dr Pedro Machado pedro.machado@ntu.ac.uk



Nottingham Trent
University

Department of Computer Science

Lecture Outline

In this lecture will cover:

- Understand neuroevolution paradigms
- Understand Deep Optimisation (DO)
- Compare gradient-based vs evolutionary methods
- Analyse hybrid Evolution + Deep RL frameworks

This lecture contributes to the following learning outcomes:

MLO2: Critically evaluate the strengths and weaknesses of different computational intelligence techniques for solving complex problems.

MLO4: Understand the ethical implications of using computational intelligence technologies and the potential impact on society.

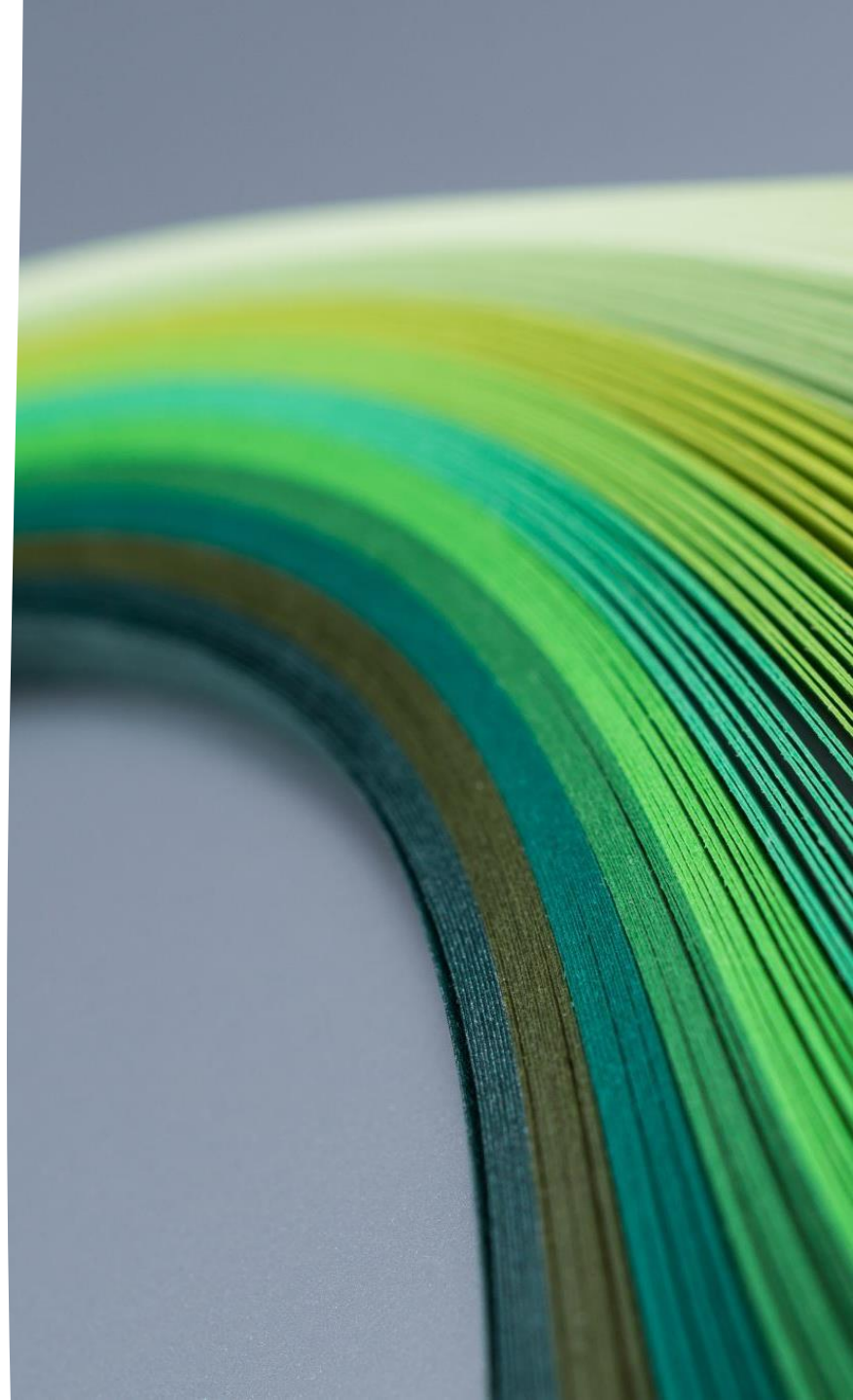
MLO5: Apply computational intelligence techniques to solve real-world problems, demonstrating practical problem-solving skills.

MLO6: Develop proficiency in programming and using computational tools for implementing and evaluating intelligent systems.

MLO8: Communicate effectively the concepts, methodologies, and results of computational intelligence research to both technical and non-technical audiences.

Why Optimisation Still Matters

- Combinatorial optimisation (CO) involves searching over large, discrete solution spaces common in scheduling, routing, and resource allocation.
- Many intelligent systems operate in high-dimensional, non-convex landscapes with numerous local optima.
- Gradient-based methods often struggle in discrete or highly structured domains.
- Deep models themselves create complex search spaces over architectures, weights, and policies.
- Effective optimisation underpins performance, robustness, and scalability in modern intelligent systems.



Two Grand Paradigms



Gradient-based deep learning optimises model parameters using backpropagation and stochastic gradient descent, leveraging differentiability for efficient local improvement.



Evolutionary algorithms (EAs) perform population-based, gradient-free search through selection and variation, making fewer assumptions about the objective landscape.



Gradient methods typically excel in smooth, continuous domains, while EAs are robust in noisy, discrete, or deceptive environments.



Policy search focuses on directly optimising decision-making strategies, often in reinforcement learning contexts.

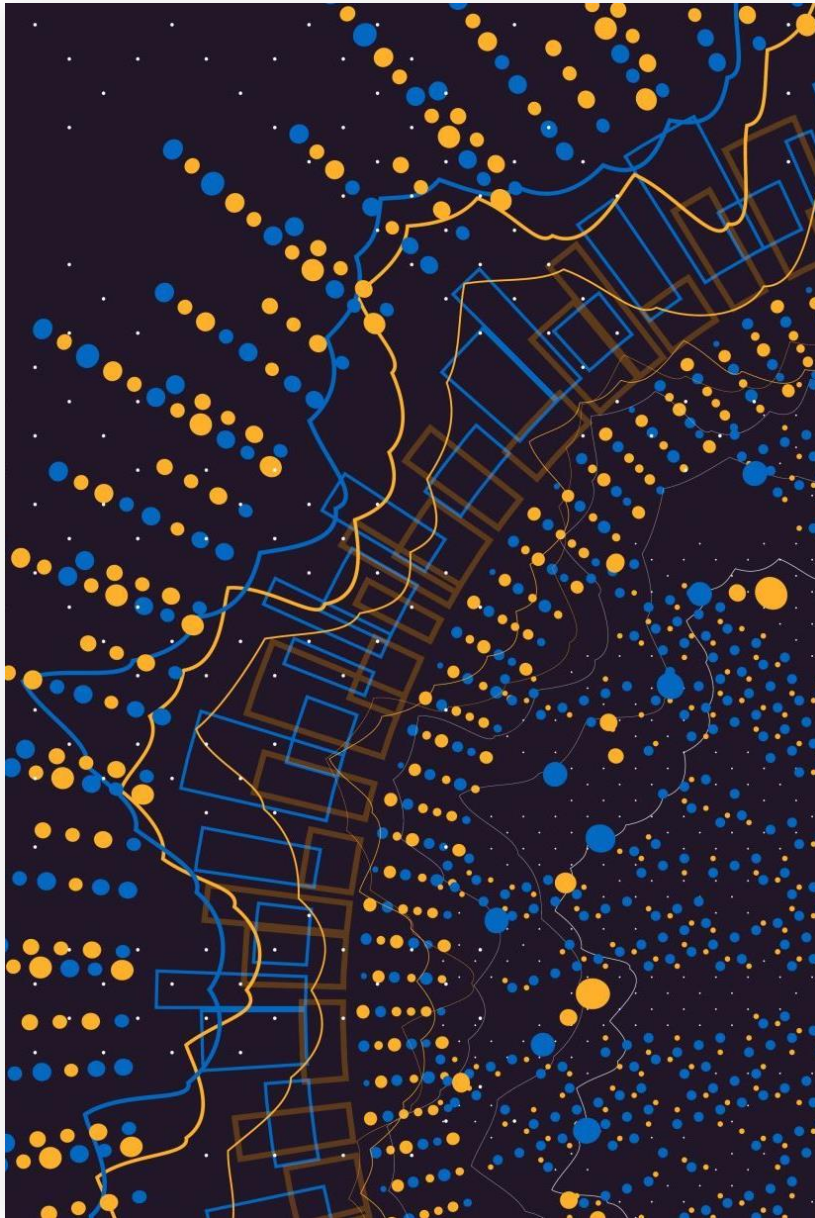


Parameter search concentrates on tuning weights or architectures to improve predictive or control performance.

Limitations of Pure Gradient Methods

- Optimisation landscapes in deep models are highly non-convex, leading to local minima and saddle points that can slow or stall convergence.
- Gradient estimates, particularly in stochastic settings, can exhibit high variance and noise, reducing stability.
- In reinforcement learning, policy gradients may provide biased or imprecise updates due to limited sampling.
- Gradient-based methods require differentiability, restricting their applicability.
- Discrete and combinatorial optimisation problems cannot be directly handled without relaxation or surrogate formulations.





Strengths of Evolutionary Methods

- Evolutionary methods are gradient-free and do not require differentiability of the objective function.
- Population-based search enables simultaneous exploration of multiple regions of the solution space.
- Selection mechanisms provide natural robustness against noisy or misleading updates.
- Evolutionary operators can traverse large, non-local moves in the search space.
- Such methods are well suited to discontinuous, discrete, and highly non-convex optimisation problems.

From Model-Building EAs to Bayesian and Deep Models



Model-based evolutionary methods replace crossover and mutation with probabilistic models of good solutions.



They learn relationships between variables and generate new candidates by sampling from this learned structure.



Bayesian Optimisation builds a statistical model of the objective function and uses it to guide where to sample next.



Both approaches shift search from blind trial-and-error to informed, model-guided exploration.



Deep models extend this by learning more complex, hierarchical structure in the problem.

What is Neuroevolution?

- Neuroevolution refers to the use of evolutionary algorithms to optimise components of neural networks.
- Evolving architectures involves searching over network topologies, layer configurations, and connectivity patterns.
- Evolving weights replaces or complements backpropagation by optimising network parameters through population-based search.
- Hybrid approaches combine evolutionary search with gradient-based learning to exploit the strengths of both paradigms.
- Neuroevolution provides a flexible framework for automated design and optimisation of deep neural networks.



Evolutionary Algorithms Refresher

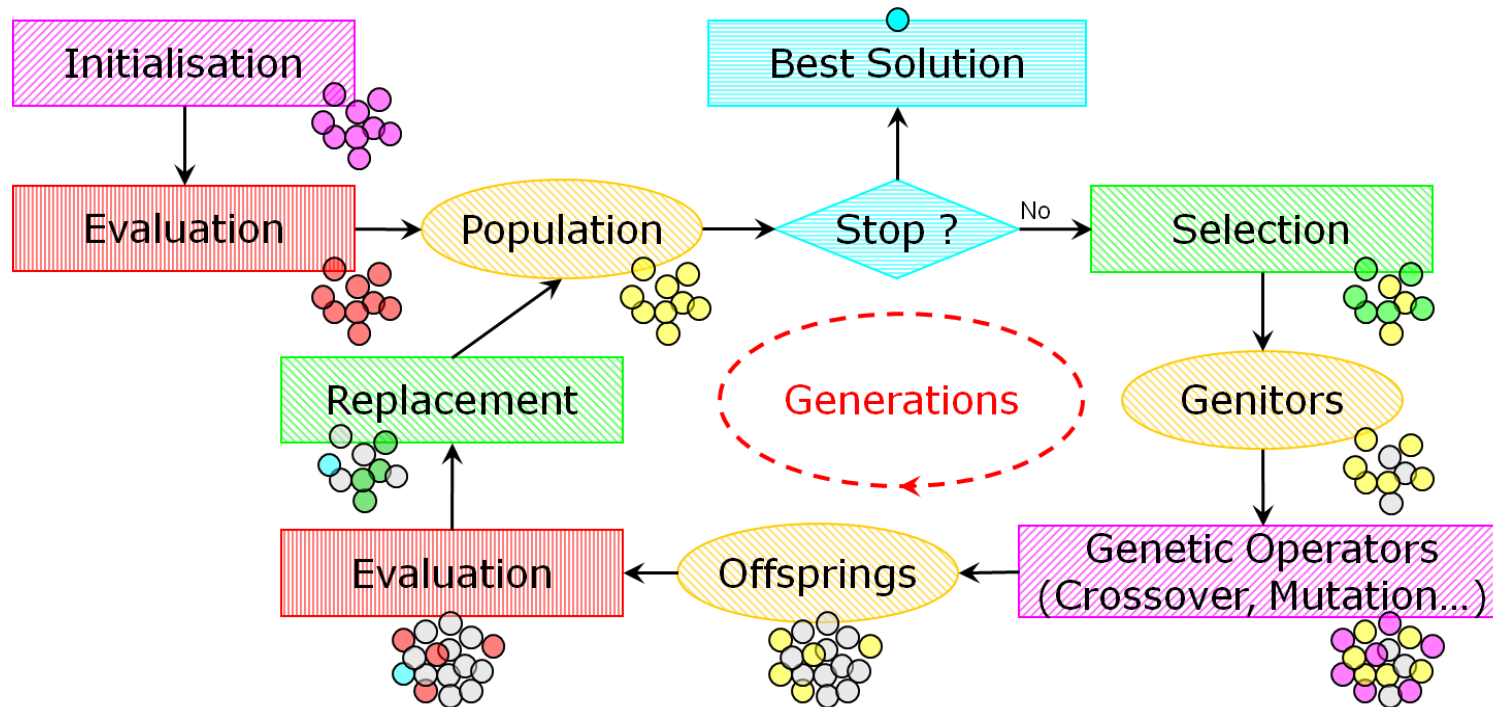
Selection prioritises higher-performing individuals, increasing the probability that beneficial traits are propagated to the next generation.

Variation introduces diversity through mutation and crossover, enabling exploration of new regions of the search space.

Mutation applies random perturbations to individuals, while crossover recombines components from multiple parents.

Fitness evaluation assigns a quantitative score to each individual based on task performance.

Iterative cycles of selection, variation, and evaluation gradually drive the population toward improved solutions.



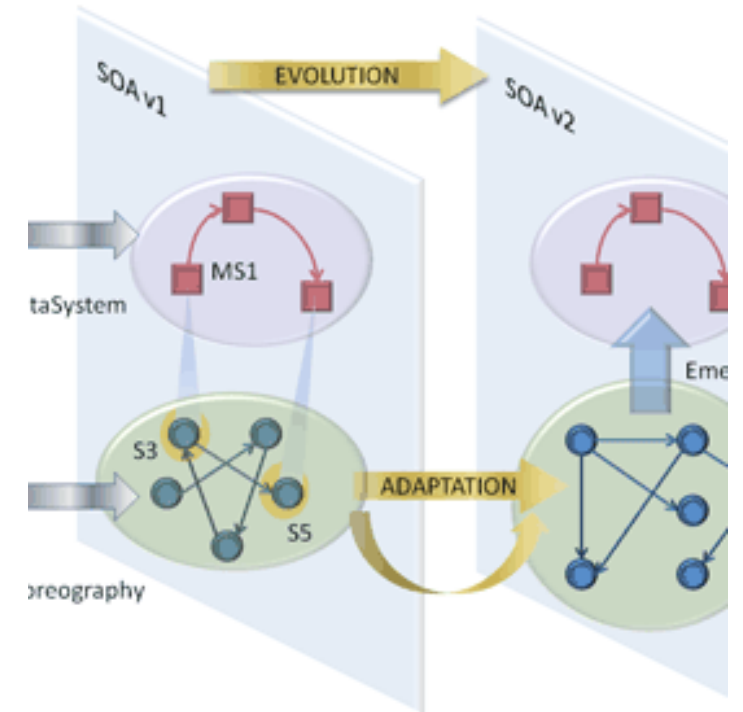
Bayesian Optimisation (BO)

- Designed for optimisation where evaluations are expensive, noisy, or lack gradients
- Builds a probabilistic surrogate model of the objective function (e.g. performance, accuracy, cost)
- Uses an acquisition function to decide where to evaluate next
- Balances:
 - Exploration (learn unknown regions)
 - Exploitation (refine promising areas)
- Iterative loop: Model $\rightarrow$ Select next candidate $\rightarrow$ Evaluate $\rightarrow$ Update belief
- Enables sample-efficient search in black-box problems
- Commonly used for:
 - Hyperparameter tuning
 - Neural Architecture Search
 - Experimental design



Evolving Architectures (NAS)

- Neural Architecture Search (NAS) aims to automatically discover effective network topologies rather than relying on manual design.
- Genetic Algorithms explore architecture space through selection, crossover, and mutation applied to encoded network structures.
- Evolution Strategies operate on continuous parameterisations and can optimise architecture-related hyperparameters using population-based gradient approximations.
- Architecture encoding may represent layer types, connectivity patterns, filter sizes, and depth.
- Evolutionary NAS enables systematic exploration of complex design spaces that are difficult to search with gradient-based methods alone.



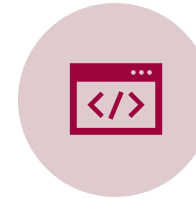
Representation Choices



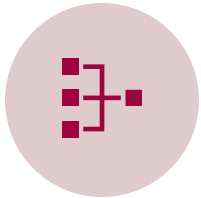
Direct encoding represents network parameters or structures explicitly, such as listing all weights or layer configurations in a genome.



Indirect or grammar-based encoding uses compact rules or generative descriptions to produce complex architectures from simpler representations.



Indirect encodings can improve scalability by capturing regularities and modular patterns.



Topology augmentation incrementally grows network structure, allowing complexity to increase during evolution.

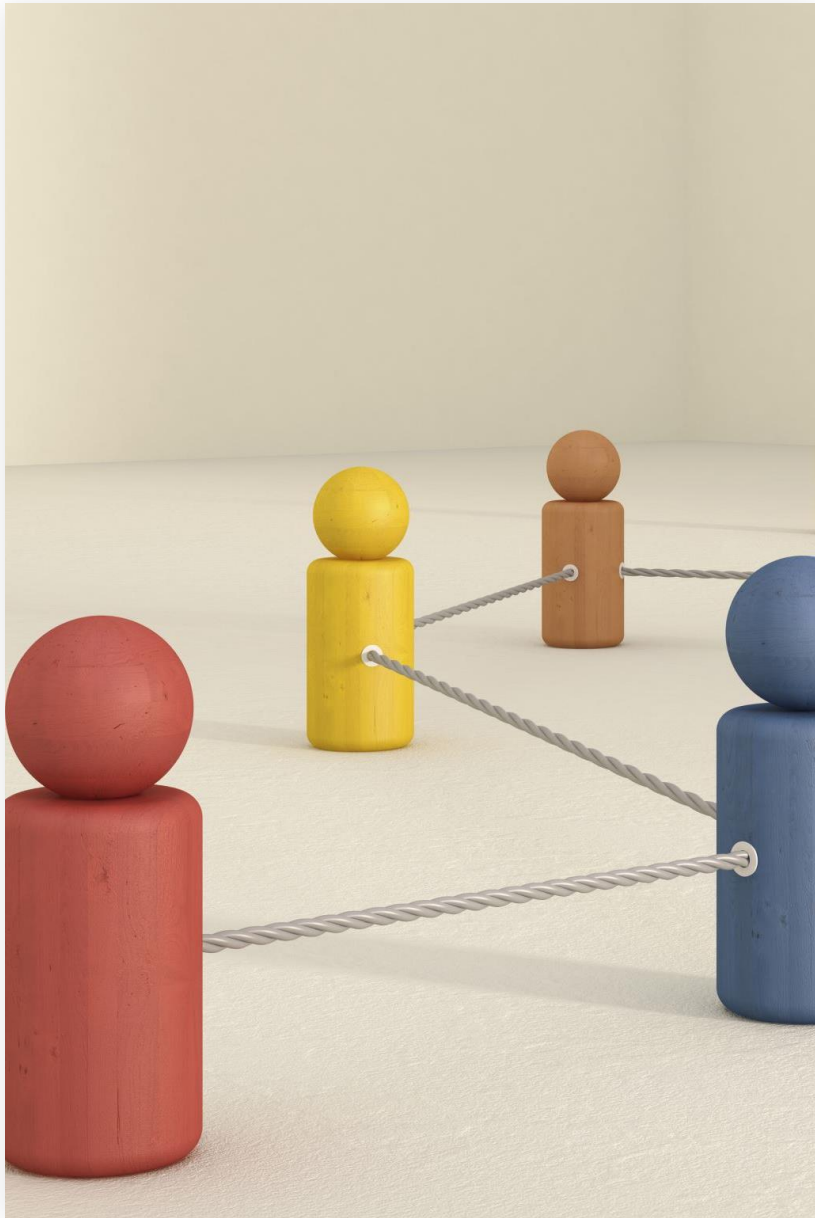


The choice of representation strongly influences search efficiency, scalability, and expressiveness.

Fitness Evaluation in DNN Evolution

- Fitness evaluation typically requires training candidate networks, which can be computationally expensive.
- Full training provides accurate performance estimates but significantly increases search time.
- Partial training reduces cost but introduces noise and uncertainty in fitness estimation.
- Surrogate fitness models approximate true performance using predictive models to accelerate evaluation.
- Balancing accuracy and computational efficiency is a central challenge in evolutionary optimisation of deep networks.





Neuroevolution for Weight Training

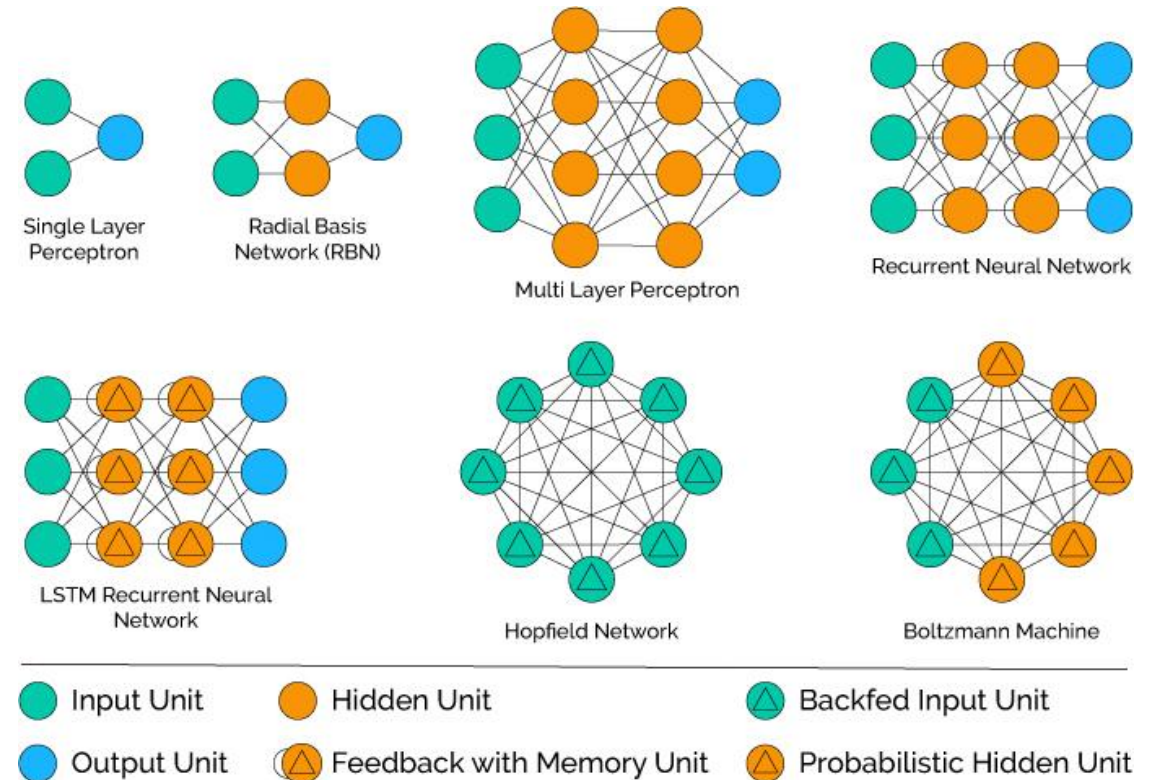
- Neuroevolution can optimise network weights directly, replacing or complementing stochastic gradient descent (SGD).
- Unlike SGD, evolutionary methods do not rely on exact gradient computation and therefore do not require differentiability.
- Population-based search explores multiple candidate weight configurations simultaneously.
- The distribution of individuals in the population provides an implicit approximation of the gradient direction.
- Such approaches can be more robust in noisy, sparse-reward, or deceptive optimisation landscapes.

Neuroevolution Vs SGD

Aspect	Neuroevolution	Stochastic Gradient Descent (SGD)
Optimisation Type	Gradient-free, population-based search	Gradient-based, single-solution iterative update
Requirement	Does not require differentiability	Requires differentiable objective function
Search Strategy	Explores multiple candidate solutions simultaneously	Updates one parameter vector using local gradient
Exploration	Naturally promotes global exploration	Primarily performs local optimisation
Robustness to Noise	More robust to noisy or sparse rewards	Can be sensitive to gradient noise and variance
Handling Discrete Variables	Naturally supports discrete and structured spaces	Requires relaxation or surrogate formulations
Convergence Behaviour	Often slower but less prone to local traps	Typically faster but may get stuck in local minima or saddle points
Parallelisation	Highly parallelisable across individuals	Parallelisation usually limited to batch computation
Computational Cost	High due to repeated evaluations	Lower per update, but may require many iterations
Typical Use Cases	Architecture search, policy search, non-differentiable tasks	Supervised learning, large-scale differentiable models

Criticisms of Neuroevolution

- Neuroevolution often incurs high computational cost due to repeated evaluation of large populations of networks.
- Training each candidate model, even partially, can be resource-intensive and time-consuming.
- Unlike gradient-based methods, evolutionary approaches typically lack strong theoretical convergence guarantees.
- Performance may depend heavily on representation choices and operator design.
- Scaling to very deep networks or large datasets remains a practical challenge without substantial parallel computing resources.

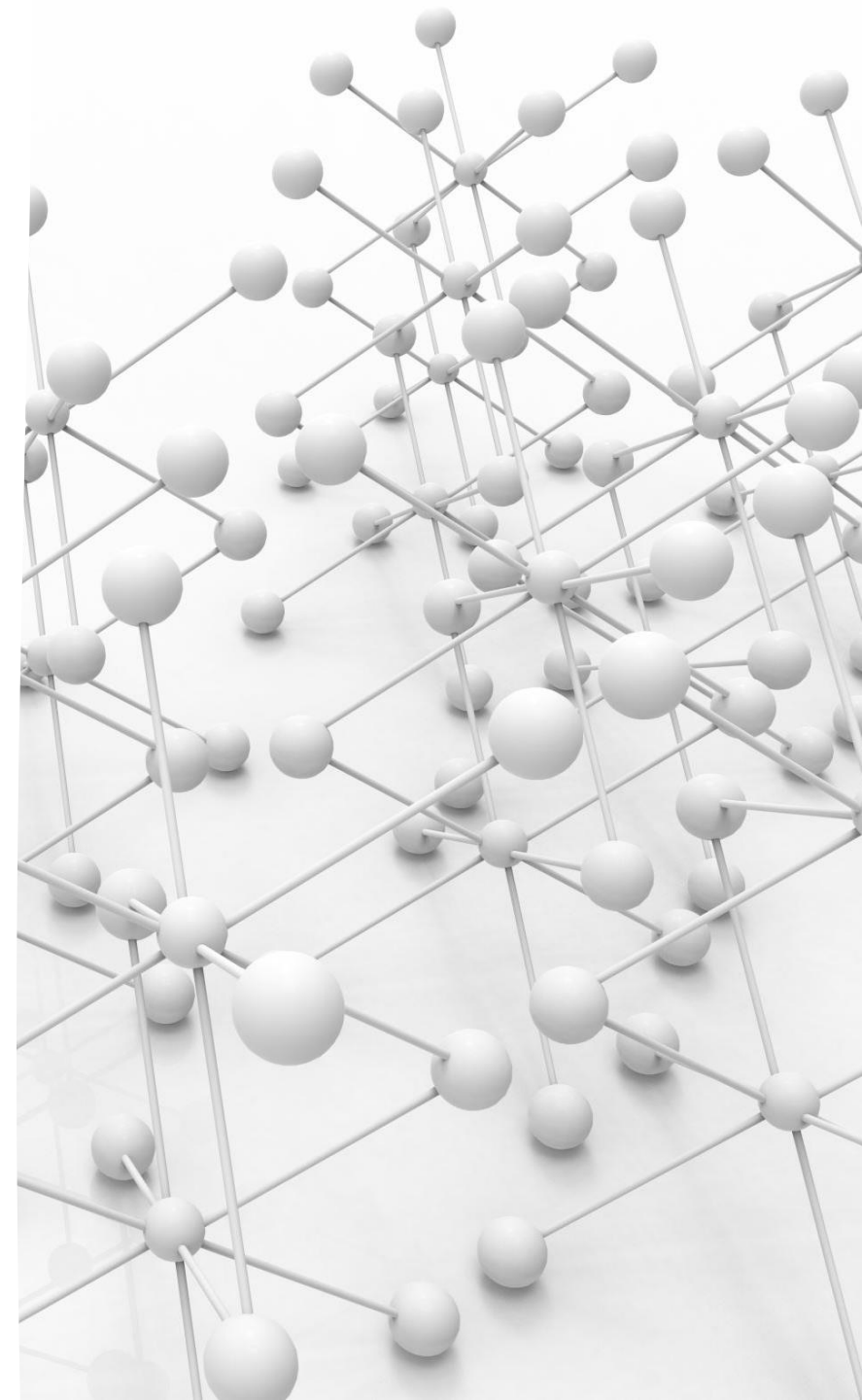


Neuroevolution vs Backpropagation

Aspect	Neuroevolution	Backpropagation
Optimisation Principle	Population-based evolutionary search	Gradient-based optimisation via chain rule
Differentiability Requirement	Not required	Required
Update Mechanism	Selection, mutation, crossover	Gradient descent updates
Search Behaviour	Global, exploratory	Local, exploitative
Handling Discrete Structures	Naturally supported	Requires relaxation or special design
Sensitivity to Saddle Points	Less sensitive due to population diversity	Can stall in flat or saddle regions
Parallelisation	Naturally parallel across individuals	Parallel mainly at batch level
Convergence Speed	Often slower per iteration	Typically faster per update
Theoretical Guarantees	Limited formal guarantees	Supported by optimisation theory
Typical Applications	Architecture search, RL, black-box optimisation	Supervised deep learning, differentiable models

Deep Architectures in Neuroevolution

- Convolutional Neural Networks (CNNs) are commonly evolved for image and spatial data tasks, including architecture depth, filter sizes, and connectivity patterns.
- Evolutionary search can optimise CNN topology as well as hyperparameters such as kernel dimensions and pooling strategies.
- Autoencoders are evolved to discover compact latent representations, particularly in unsupervised or feature-learning settings.
- Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are evolved for sequential and time-dependent tasks.
- Neuroevolution enables automated configuration of complex deep architectures beyond manual design heuristics.



Bayesian Optimisation for Multi-Objective Search

- Real-world model design rarely involves a single objective.
- Performance must often be balanced against constraints such as:
 - Model size
 - Latency
 - Energy use
 - Robustness
- Bayesian Optimisation can be extended to handle multiple objectives.
- Instead of finding one “best” solution, it learns the trade-offs between competing goals.
- The aim becomes efficient exploration of the space of possible compromises.
- A structured way to discover high-performing solutions under practical constraints.
- Leading naturally into Pareto-based neuroevolution



Multi-objective Neuroevolution

- Multi-objective neuroevolution optimises multiple criteria simultaneously rather than a single performance metric.
- A common trade-off is accuracy versus model complexity, where smaller models may be preferred for efficiency.
- Additional objectives can include energy consumption, inference latency, or robustness.
- Pareto optimisation identifies a set of non-dominated solutions representing different trade-offs.
- The resulting Pareto front enables informed selection based on deployment constraints and system requirements.

Open Research Directions

- Surrogate-assisted evolutionary algorithms use predictive models to approximate the true objective function, reducing the need to fully train and evaluate every candidate network; a surrogate is a secondary model that estimates performance based on previously evaluated solutions.
- Such surrogate models can significantly reduce computational cost, particularly in expensive deep learning settings.
- Hybrid stochastic gradient descent and evolutionary algorithms combine gradient-based fine-tuning with population-based global exploration.
- Careful coordination between gradient updates and evolutionary variation remains an open design challenge.
- Fitness landscape analysis seeks to understand the structure of optimisation surfaces, including modality, ruggedness, and neutrality, to better guide algorithm design.

Transition to Deep Optimisation

- Earlier approaches focused on evolving neural networks to improve predictive or control performance.
- The emphasis was on searching over architectures or weights within the model itself.
- Deep Optimisation shifts the perspective from evolving networks to using networks as models of problem structure.
- Neural networks are trained to learn latent representations that capture dependencies within candidate solutions.
- The learned representation is then used to guide and restructure the optimisation search process.

Core Idea of Deep Optimisation

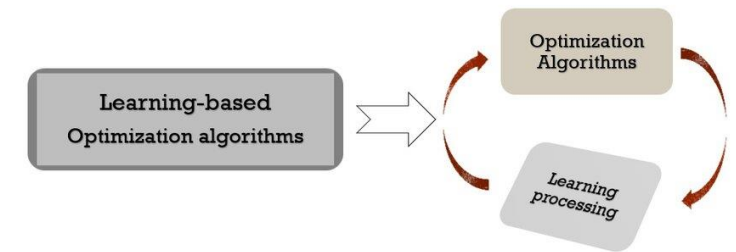
- Deep Optimisation uses a neural network model to learn structural regularities from high-quality candidate solutions.
- Instead of applying random variation, the algorithm performs model-informed variation guided by the learned representation.
- Changes are made in a latent space and decoded into coordinated modifications of multiple problem variables.
- This mechanism enables structured, non-local moves in the search space.
- The approach is designed to uncover and exploit deep hierarchical structure within combinatorial optimisation problems.

DO vs Traditional MBOAs

Aspect	Model-Building Optimisation Algorithms (MBOAs)	Deep Optimisation (DO)
Core model	Typically shallow probabilistic or dependency models (e.g., Bayesian networks, linkage trees, dependency matrices)	Deep autoencoder (multi-layer neural network) that learns compressed representations
Structure captured	Often limited in depth; commonly emphasises lower-order or explicitly modelled dependencies	Targets hierarchical, multi-level (“deep”) structure via successive representation learning
How new solutions are produced	Frequently samples complete candidates from the learned model (model-informed generation)	Improves a solution through iterative model-informed variation by changing latent variables and decoding back
Search neighbourhood	Adapted by learned dependencies, but generally within a single modelling level	Neighbourhood is repeatedly rescaled as deeper representations are learned across layers
Learning signal	Learns from promising/selected solutions to infer linkage and correlations	Learns from distributions of locally optimal solutions produced at each representation level
Intended advantage	Efficient recombination and preservation of learned linkages	Coordinated, non-local moves through latent-space edits that correspond to organised multi-variable changes

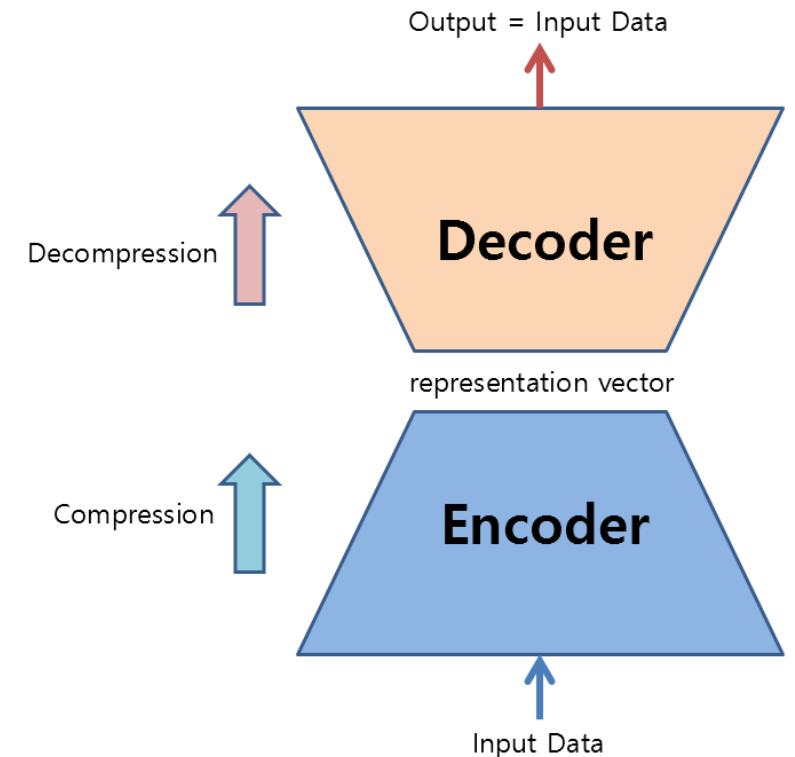
Deep Optimisation Algorithm Overview

- Deep Optimisation consists of two interlocked processes: a solution optimisation loop and a model optimisation loop.
- The solution optimisation loop performs iterative local search using model-informed variation to improve a candidate solution.
- Changes to the solution are accepted or rejected based on fitness evaluation.
- The model optimisation loop updates the neural network parameters using improved solutions as training data.
- The two loops influence each other, as improved solutions refine the model, and the refined model restructures future search.



Autoencoder as Structure Learner

- An autoencoder is used to learn a compressed representation of candidate solutions.
- The encoder transforms the original solution variables into a lower-dimensional latent representation.
- This compression captures statistical regularities and dependencies among variables.
- The decoder reconstructs the original solution from the latent variables.
- The latent space serves as a structured search space for coordinated and informed variation.



Model-Informed Variation (MIV)

01

Model-informed variation operates in the latent space learned by the autoencoder rather than directly in the original solution space.

02

Small changes are applied to latent variables instead of individual problem variables.

03

The modified latent representation is passed through the decoder.

04

Decoding produces coordinated, multi-variable changes in the original solution.

05

Such structured variation increases the likelihood of adaptive, large-scale improvements in complex optimisation landscapes.

Learning from Locally Optimal Solutions

Deep Optimisation uses a simple hill-climbing procedure to generate locally optimal solutions from different starting points.

Hill-climbing iteratively accepts fitness-improving changes until no further local improvement is possible.

The collection of such local optima forms a distribution that reflects partial structure in the problem.

This distribution provides training data for the autoencoder.

Learning from these solutions enables the model to extract recurring patterns and dependencies embedded in the fitness landscape.

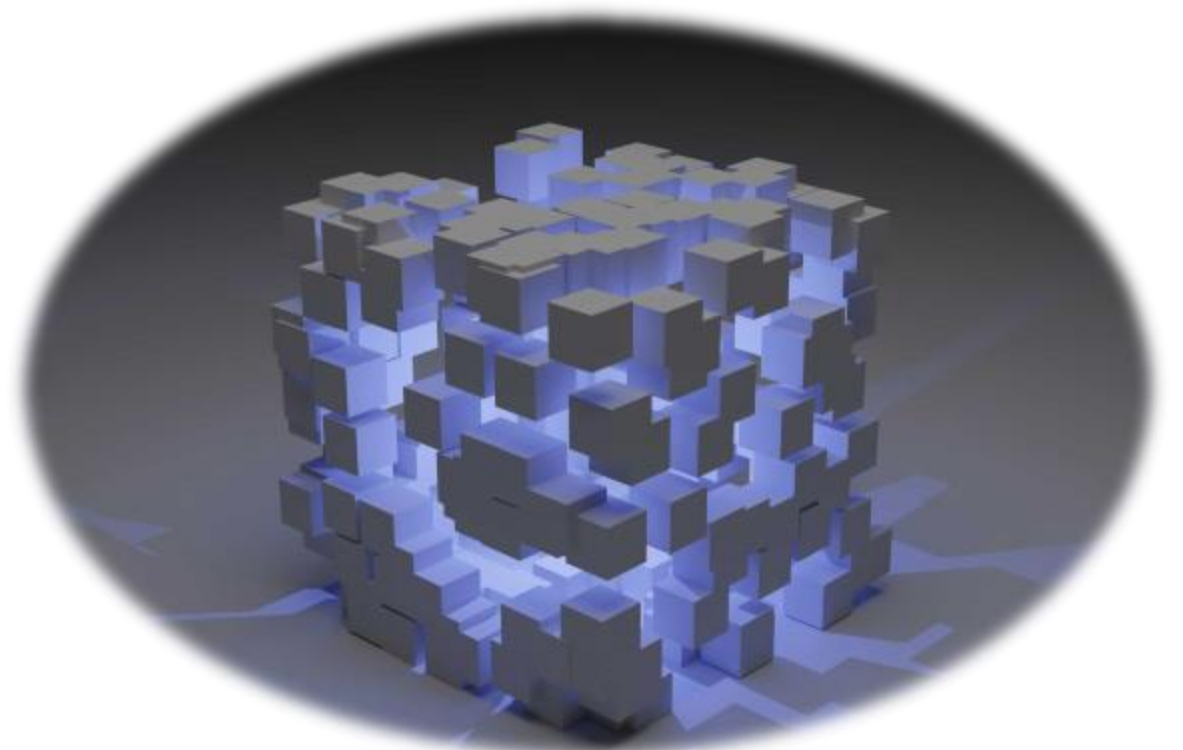
Evolutionary Transitions in Individuality Analogy

- Deep Optimisation draws inspiration from evolutionary transitions in individuality, where lower-level units combine to form higher-level entities.
- Each learned representation layer can be viewed as creating a new level of organisation.
- Multi-level representations allow search to operate on progressively more abstract structures.
- A single latent change may correspond to coordinated changes across many original variables.
- These higher-order evolutionary units enable more efficient exploration of complex problem spaces.



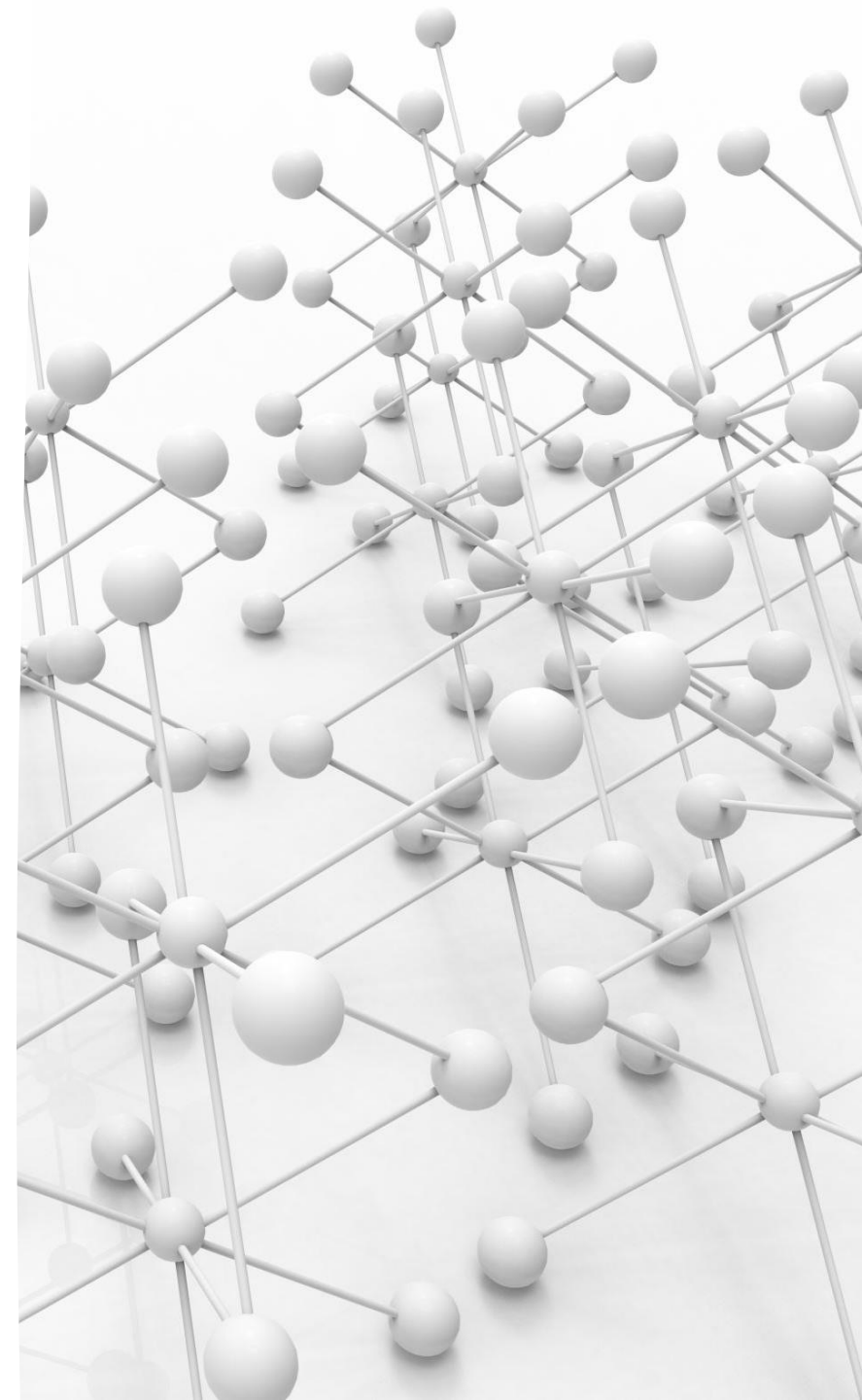
Layerwise Learning in Deep Optimisation

- Deep Optimisation builds representations incrementally through layerwise learning.
- Each layer is trained to capture structure present in solutions optimised at the previous level.
- The process progressively induces deeper, more abstract representations of the problem.
- With each added layer, the effective search space is rescaled to operate at a higher level of organisation.
- Such representation rescaling enables coordinated variation over increasingly structured units.

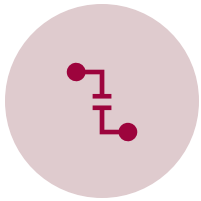


Transition Mechanism

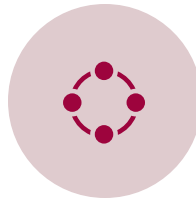
- The first step is to discover how lower-level variables combine by analysing patterns in locally optimal solutions.
- The second step is to compress these combinations into a latent representation using a neural network model.
- This compression induces higher-order units that capture coordinated variable interactions.
- The final step is to rescale variation so that search operates in the transformed representation space.
- Each transition enables more structured and potentially larger moves across the optimisation landscape.



Deep Structure in Optimisation



Many optimisation problems contain dependencies that extend beyond simple pairwise interactions between variables.



Higher-order interactions may involve coordinated relationships among multiple variables simultaneously.



Shallow models often struggle to represent such complex dependencies effectively.



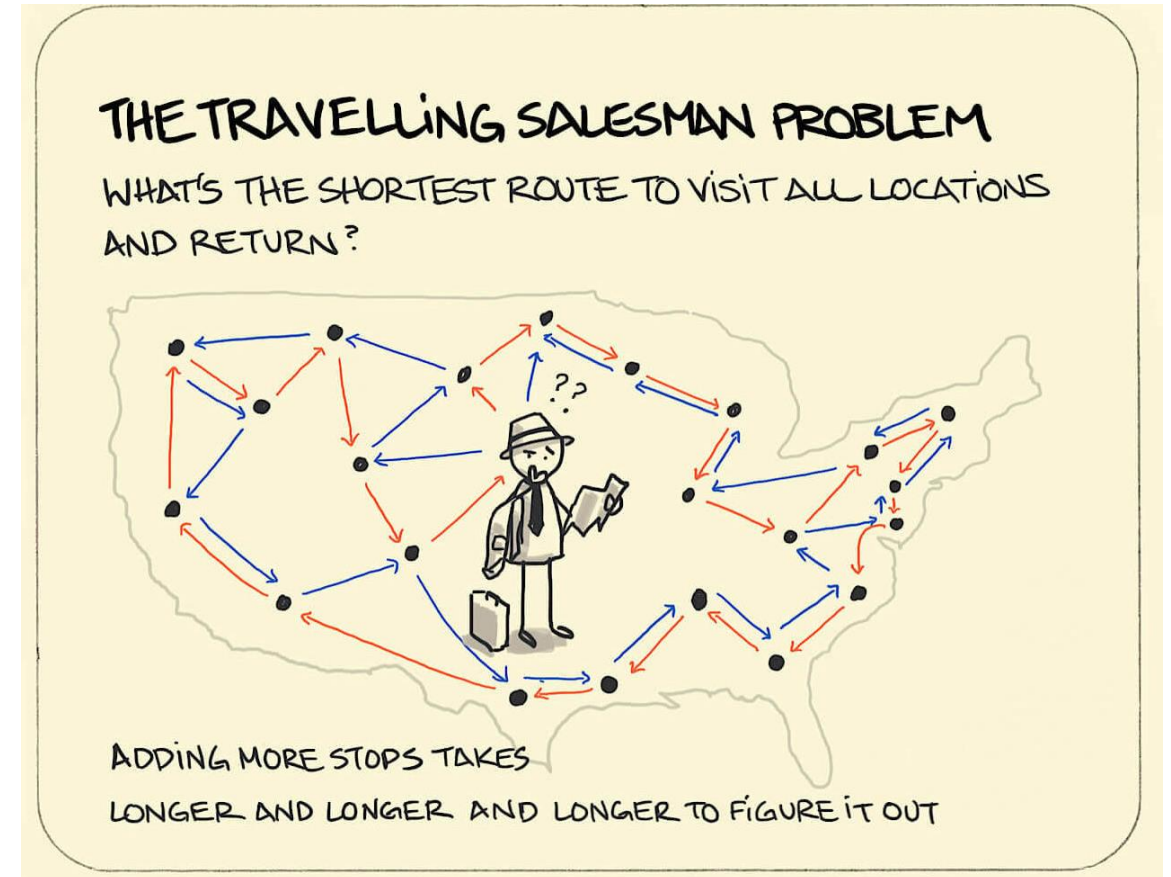
Hierarchical dependencies emerge when lower-level combinations form structured modules at higher levels.



Capturing this deep structure enables more efficient and informed exploration of combinatorial search spaces.

Deep Optimisation for the Travelling Salesman Problem

- The Travelling Salesman **Problem** uses a non-binary representation, where solutions are permutations rather than bit strings.
- Deep Optimisation adapts to this structure by learning patterns within permutation-based candidate solutions.
- The autoencoder captures regularities in route configurations that correspond to high-quality tours.
- Model-informed variation generates coordinated modifications in the latent space, which decode into structured route changes.
- The approach demonstrates that Deep Optimisation can handle non-binary encodings and problems where infeasible solutions may arise during search.



Polynomial vs Exponential Scaling

Aspect	Deep Optimisation	Classical Model-Building Optimisation
Scaling behaviour	Polynomial growth with problem size	May exhibit exponential growth
Source of efficiency	Learns hierarchical structure	Limited structure exploitation
Handling dependencies	Captures deep variable relationships	Struggles with deep dependencies
Computational effort	Grows more gradually	Can grow rapidly
Representation role	Learnt representations improve scaling	Shallow modelling limits scalability
Impact on combinatorial problems	Alters scaling behaviour	Often constrained by complexity growth

Exploration vs Exploitation in Deep Optimisation

Element	Exploration	Exploitation
Role in search	Discover new promising regions	Refine known good solutions
Learning behaviour	Encouraged by faster adaptation	Supported by stable updates
Learning rate effect	Higher rates promote discovery	Lower rates improve refinement
Risk	Overfitting to recent solutions	Slow structural learning
Model update	Rapid response to new optima	Gradual consolidation
Search impact	Maintains diversity	Improves solution quality
Balance requirement	Needed to avoid premature convergence	Needed to ensure progress

Strengths of Deep Optimisation

- Deep Optimisation learns deep modularity by uncovering hierarchical relationships among problem variables.
- The autoencoder induces higher-order units that reflect coordinated patterns in high-quality solutions.
- Such modular representations enable structured, multi-variable variation.
- The search neighbourhood is adaptively restructured as deeper representations are learned.
- This dynamic rescaling improves the ability to traverse complex and deceptive optimisation landscapes.



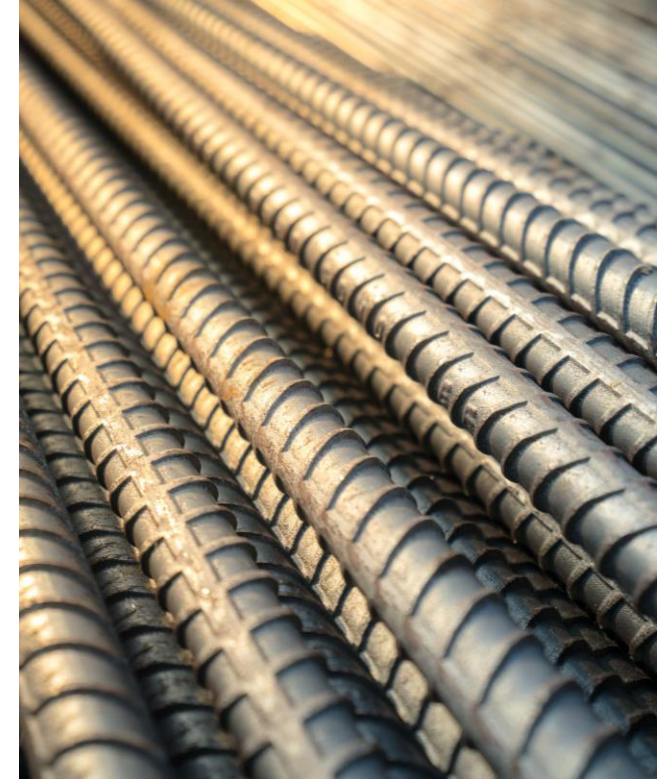
Limitations of Deep Optimisation

- Training the autoencoder introduces additional computational overhead compared to purely heuristic search methods.
- The quality of learned structure depends on the diversity and representativeness of locally optimal solutions.
- If local optima are poor or highly similar, the model may fail to capture useful dependencies.
- Layerwise learning increases algorithmic complexity and may require careful tuning.
- Performance can degrade if the learned representation misguides variation rather than improving it.



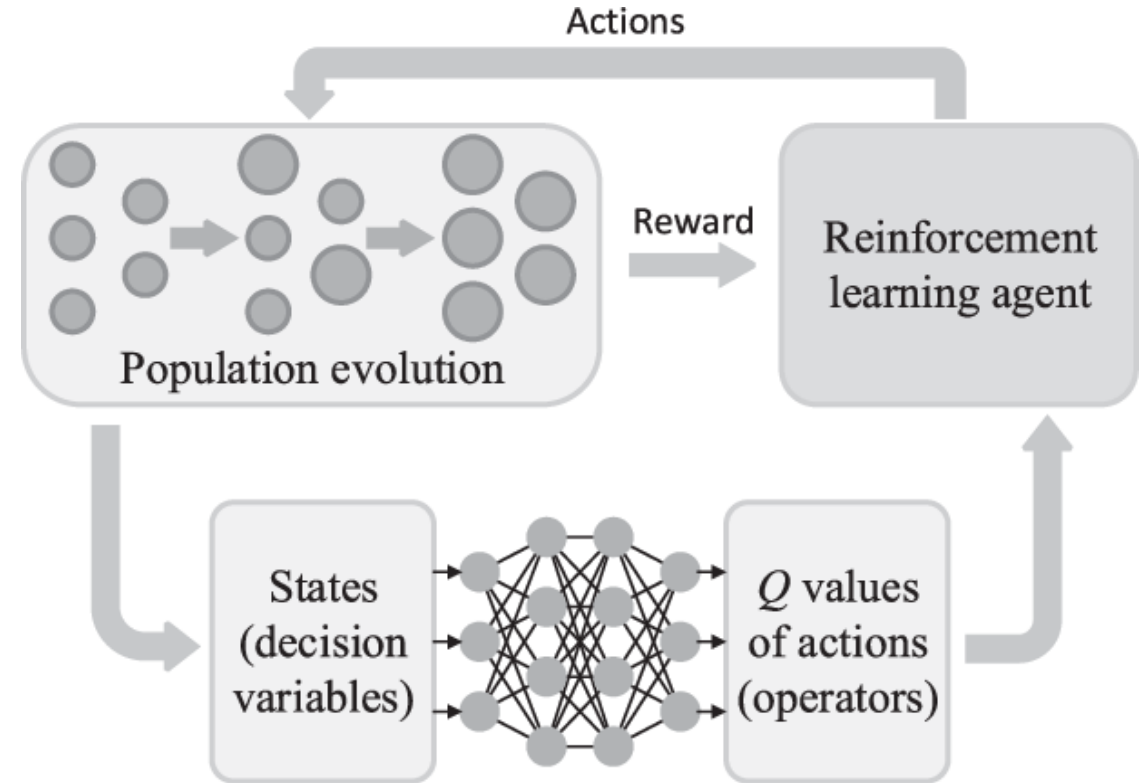
Why Combine Evolution and Reinforcement Learning?

- Policy gradient methods can suffer from high variance in gradient estimates, especially in sparse or noisy reward settings.
- Evolutionary selection evaluates full policies and discards poorly performing updates, reducing the impact of misleading gradients.
- Combining both approaches can stabilise learning by filtering detrimental parameter updates.
- Evolution introduces broader exploration that complements local gradient refinement.
- The hybrid approach aims to balance global search with efficient policy improvement.

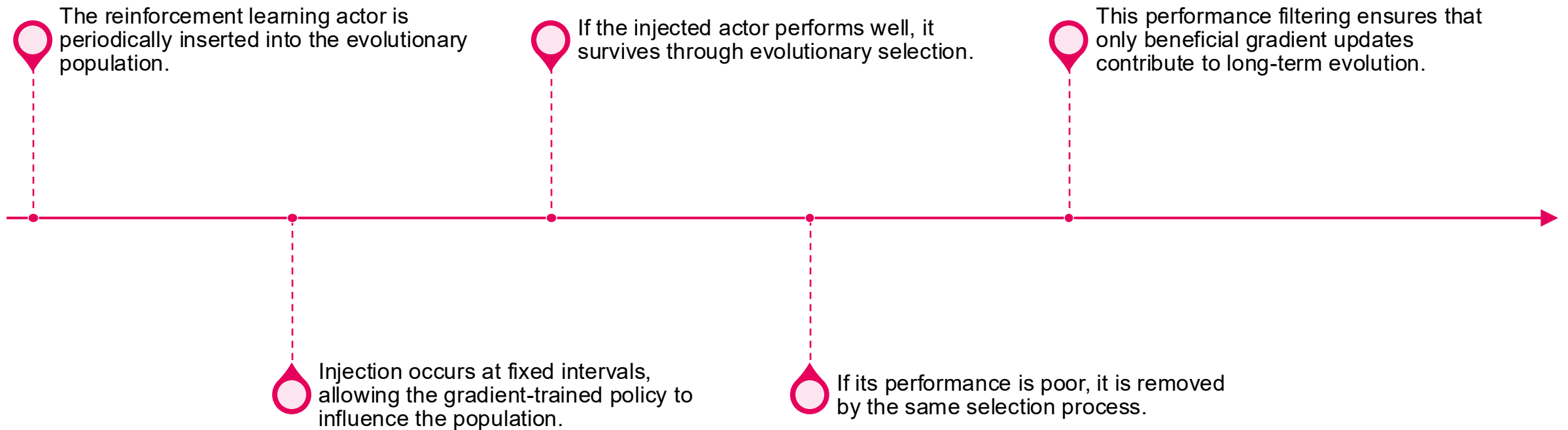


Evolutionary Reinforcement Learning (ERL)

- Evolutionary Reinforcement Learning combines a population-based evolutionary search with a gradient-based reinforcement learning agent.
- A reinforcement learning actor is periodically injected into the evolutionary population.
- The injected policy competes with evolved individuals under the same selection mechanism.
- Evolutionary selection retains higher-performing policies and discards weaker ones.
- The approach allows gradient-refined policies to accelerate evolution while preserving global exploration.



Actor Injection Mechanism



CEM-RL and Variants

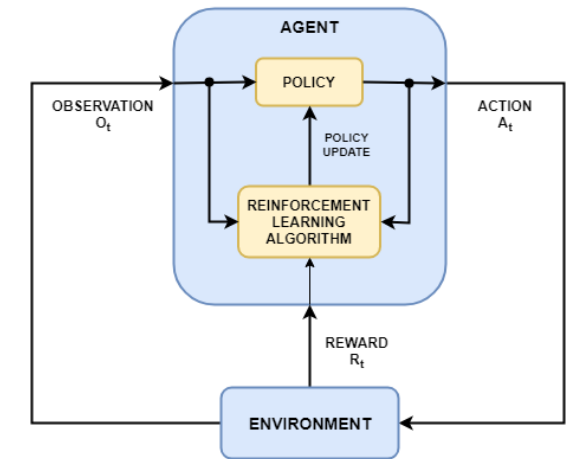
- CEM-RL combines the Cross-Entropy Method (CEM), a population-based optimisation technique, with policy gradient reinforcement learning.
- The Cross-Entropy Method maintains a probability distribution over policy parameters and updates it using elite-performing individuals.
- Policy gradient methods refine selected policies through gradient-based improvement using sampled trajectories.
- The evolutionary distribution update provides global exploration, while gradient updates provide local exploitation.
- This combination aims to improve stability and performance by integrating distributional search with policy gradient refinement.

$$D(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_j y_j \ln \hat{y}_j$$

The diagram illustrates the Cross-Entropy Method (CEM) formula. It shows two probability distributions: a target distribution $\hat{\mathbf{y}}$ (represented by a vector $\begin{bmatrix} 0.1 \\ 0.5 \\ 0.4 \end{bmatrix}$) and a current distribution $\mathbf{y}$ (represented by a vector $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$). The formula $D(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_j y_j \ln \hat{y}_j$ is shown, with arrows indicating the relationship between the distributions and the formula.

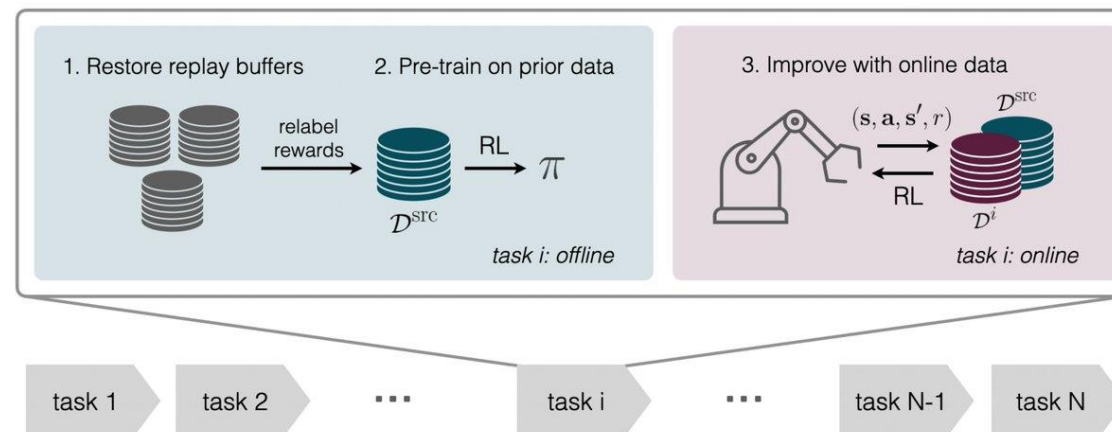
Evolution Strategies and Reinforcement Learning Hybrids

- Evolution Strategies (ES), a population-based black-box optimisation method, can be combined with reinforcement learning algorithms to improve policy search.
- One approach integrates Evolution Strategies with Soft Actor-Critic (SAC), a reinforcement learning algorithm that optimises stochastic policies with entropy regularisation.
- Evolution Strategies provide global exploration by sampling and evaluating policy parameter perturbations.
- Reinforcement learning methods such as Soft Actor-Critic refine policies using gradient-based updates from replayed experience.
- Finite-difference reinforcement learning approximates gradient information by evaluating parameter perturbations, bridging evolutionary search and gradient estimation.



Replay Buffers in Hybrids

- A replay buffer stores past interactions in the form of state, action, reward, and next-state transitions.
- In hybrid frameworks, experience collected by evolutionary individuals can be shared through a common replay buffer.
- Reinforcement learning components use this shared experience to perform gradient-based updates.
- Experience sharing increases data efficiency by reusing trajectories across multiple learning steps.
- Such mechanisms improve training stability and reduce variance in policy updates.



Diversity-Promoting Hybrids



Diversity-promoting hybrids explicitly encourage variation among policies rather than optimising performance alone.



Behavioural diversity measures differences in policy actions or trajectories, not just parameter values.



Maintaining diverse behaviours increases exploration of the environment and policy space.



Such diversity reduces the risk of premature convergence to suboptimal strategies.



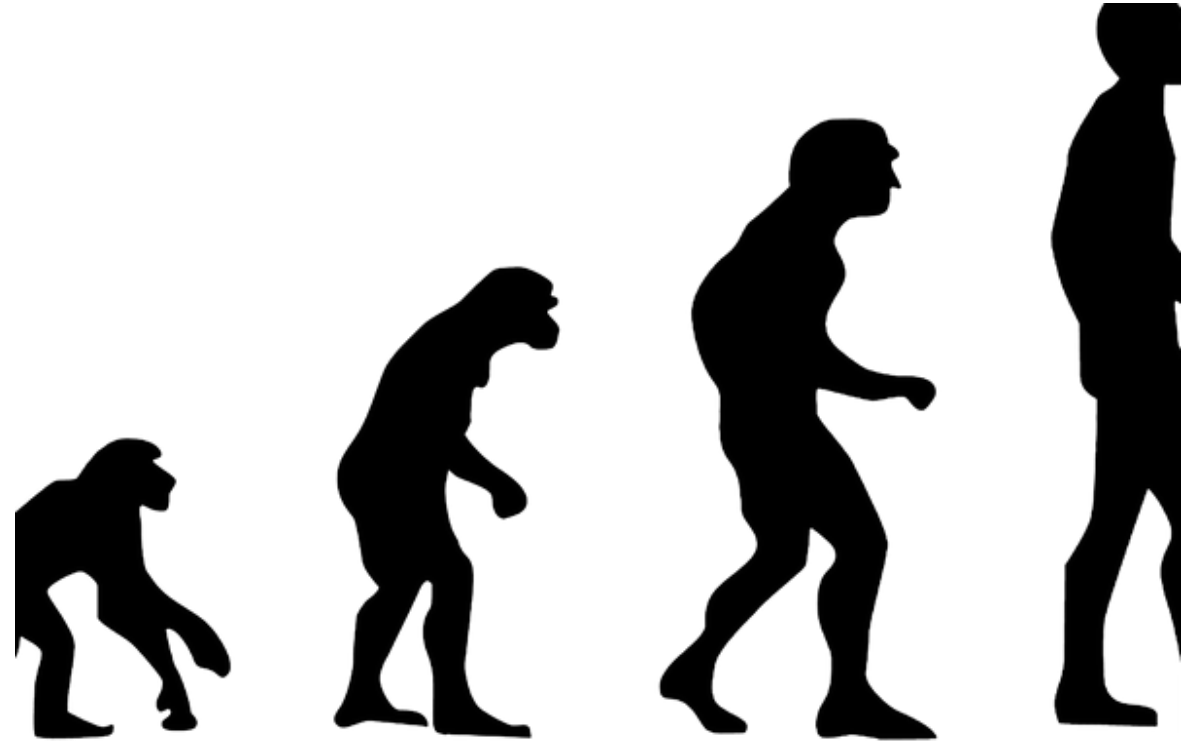
Integrating diversity objectives with gradient-based learning improves robustness in sparse or deceptive reward settings.

Policy vs Action Evolution

Aspect	Policy Evolution	Action Evolution
Optimisation Target	Full policy parameters (e.g., neural network weights)	Individual actions in given states
Evaluation Scope	Entire trajectories generated by the policy	Step-level action performance
Generalisation	Learns strategies applicable across many states	Often state-specific decision optimisation
Computational Cost	Higher per evaluation due to full rollout	Potentially lower per decision but repeated frequently
Search Space	Parameter space of the policy	Action space at each decision point
Scalability	Scales to complex tasks with structured policies	May struggle in high-dimensional or long-horizon tasks
Typical Use Case	Reinforcement learning and control problems	Online planning or adaptive decision selection

Hyperparameter Evolution in Reinforcement Learning

- Hyperparameter evolution applies evolutionary search to tune configuration parameters of reinforcement learning algorithms.
- Learning rates can be evolved to balance convergence speed and stability.
- Additional parameters such as discount factors, exploration coefficients, and update frequencies may also be optimised.
- Architecture search can evolve the structure of policy or value networks used in reinforcement learning.
- Such evolutionary tuning reduces manual trial-and-error and adapts configurations to specific tasks or environments.



Comparative Strengths

- Evolutionary algorithms excel at global exploration by maintaining diverse populations and performing large, non-local parameter changes.
- Population-based search reduces sensitivity to deceptive gradients and local optima.
- Reinforcement learning methods provide efficient local fine-tuning through gradient-based updates.
- Gradient refinement allows rapid improvement once promising regions of policy space are identified.
- Combining both approaches leverages broad exploration with precise local optimisation.





Failure Modes

- Catastrophic forgetting can occur when gradient-based updates overwrite previously learned useful behaviours.
- In hybrid systems, evolutionary variation may disrupt well-performing policy structures.
- Genetic operator misalignment arises when mutation or crossover breaks functional network structures.
- Poor coordination between evolutionary and gradient updates can introduce instability.
- Careful design of variation operators and update schedules is necessary to avoid destructive interference.

When to Use Which Hybrid?

Scenario	Evolution-Dominant Hybrid	Reinforcement Learning-Dominant Hybrid	Balanced Hybrid Approach
Sparse rewards	Strong exploration to discover rare reward signals	May struggle due to weak gradient signal	Evolution for discovery, reinforcement learning for refinement
Deceptive landscapes	Filters misleading updates through population selection	Gradients may converge to suboptimal regions	Evolution for robustness, reinforcement learning for fine-tuning
High-dimensional policies	Maintains diversity across parameter space	Efficient local updates but risk of local traps	Evolution for diversity, reinforcement learning for precision
Noisy environments	Robust to noisy fitness evaluations	Sensitive to gradient variance	Evolution stabilises updates while reinforcement learning improves efficiency
Well-shaped reward functions	Slower convergence	Fast convergence through gradient optimisation	Reinforcement learning dominant with evolutionary regularisation

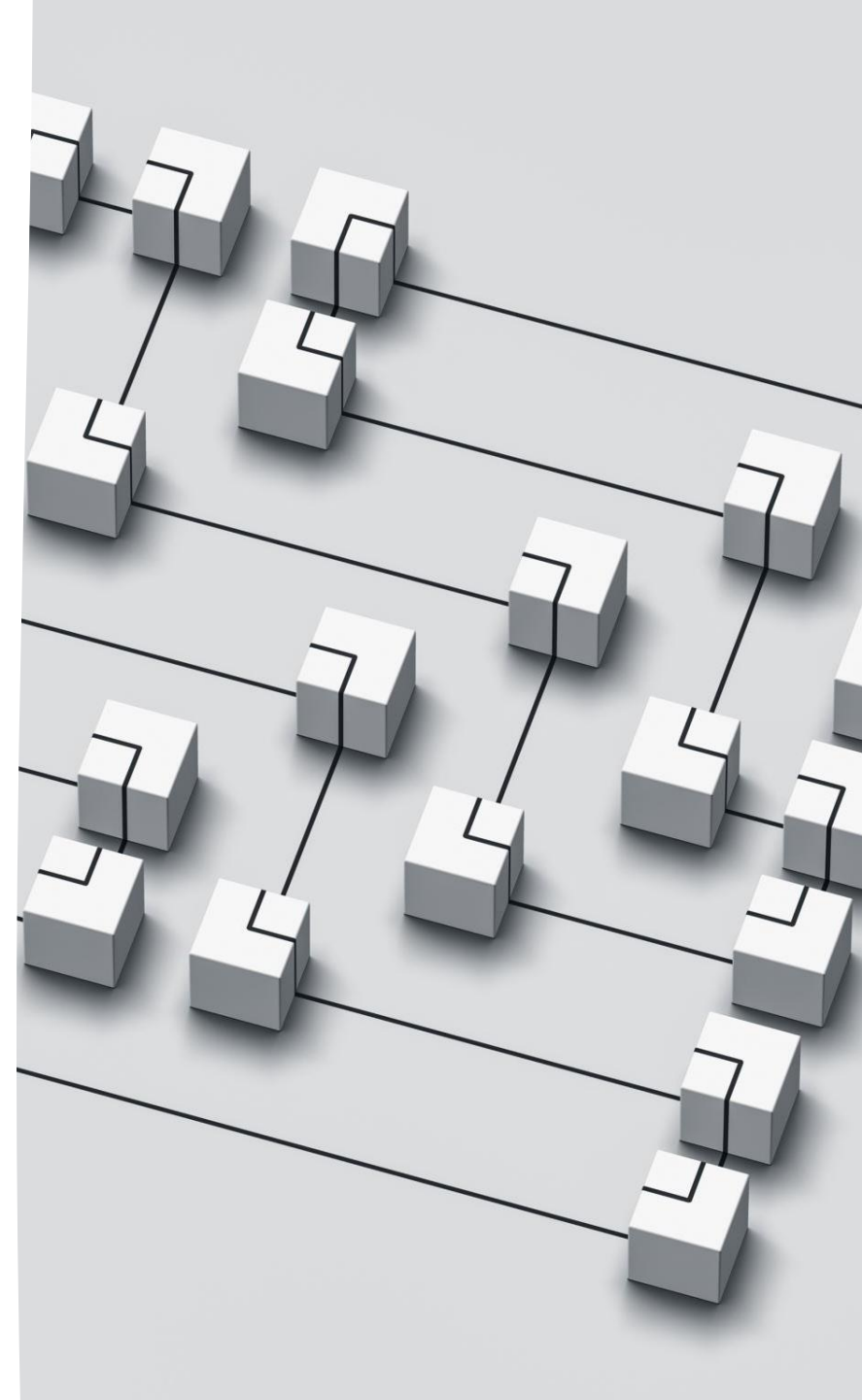
Representation as Evolutionary Unit

- Classical evolutionary search often treats individual bits or variables as the basic units of variation.
- As structure is discovered, these units can form modules that function as coordinated building blocks.
- Higher-level features emerge when modules combine into structured patterns.
- Deep Optimisation formalises this progression from bits to modules to features through hierarchical representation learning.
- The deep latent space becomes the evolutionary substrate, where variation acts on abstract, higher-order units rather than raw variables.



Deep Structure and Intelligent Systems

- Deep structure enables compositionality, where complex behaviours arise from the combination of simpler components.
- Compositional representations improve reuse and generalisation across tasks.
- Hierarchical abstraction allows systems to reason and act at multiple levels of granularity.
- Lower-level features support higher-level concepts through layered organisation.
- Intelligent systems benefit from such structure because optimisation and learning become more scalable and interpretable.



Intelligent Systems Perspective



Edge Artificial Intelligence refers to deploying learning and optimisation algorithms directly on resource-constrained devices.



Such settings require efficient, robust optimisation methods capable of operating with limited data and computation.



Autonomous systems must continuously adapt their policies in dynamic and uncertain environments.



Adaptive control integrates learning and optimisation to adjust system behaviour in real time.



Evolutionary and representation-based optimisation approaches provide flexible tools for resilient and scalable intelligent systems.

Practical Guidelines

Scenario / Requirement	Prefer Neuroevolution	Prefer Deep Optimisation	Prefer Hybrid (Evolution + Gradient / RL)
Objective Function	Non-differentiable or black-box	Structured combinatorial problems with hidden hierarchy	Partially differentiable with complex exploration needs
Problem Structure	Architecture or hyperparameter search	Deep hierarchical dependencies between variables	Policy learning with structured exploration
Reward Signal	Sparse or deceptive	Implicit structure discoverable from local optima	Sparse rewards with need for refinement
Computational Budget	Large parallel compute available	Moderate budget with structured learning	Moderate to high budget requiring both exploration and efficiency
Search Goal	Global exploration over parameters	Learning latent structure to reshape search space	Balance between global exploration and local fine-tuning
System Type	Neural architecture search, black-box control	Combinatorial optimisation tasks	Reinforcement learning and autonomous control
Scalability Needs	Parallel hardware available	Problems with exploitable modularity	High-dimensional policies requiring stability and precision

Key takeaways

- Neuroevolution focuses on evolving neural networks by searching over architectures, weights, or hyperparameters using population-based methods.
- Deep Optimisation shifts emphasis from evolving networks to learning latent structure that reshapes and guides the search process.
- Hybrid approaches combine evolutionary exploration with gradient-based efficiency to balance global and local optimisation.
- Representation learning plays a fundamental role in improving scalability and search effectiveness.
- Multi-level representations enable coordinated variation and are central to intelligent optimisation in complex systems.

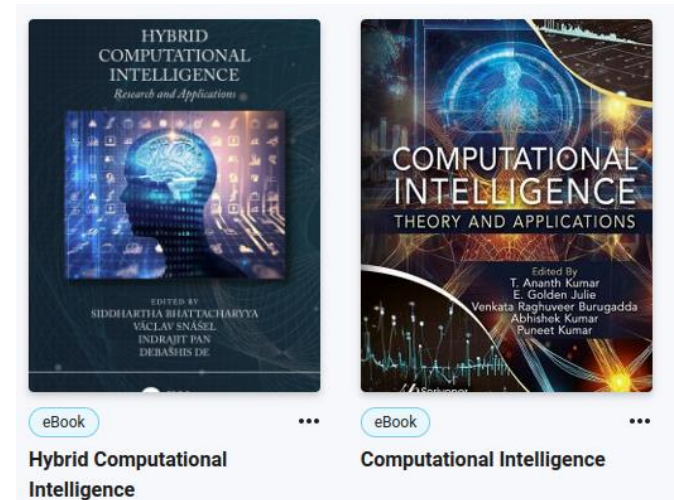


Discussion Questions

- Q1. If representation shapes the search space, what kinds of problem structures are most likely to benefit from deep, hierarchical modelling? Can you think of real-world examples?
- Q2. How might we decide dynamically when to rely more on population-based exploration versus gradient-based refinement during training? Could this balance adapt over time?

References

- [1] Caldwell, J.R., Watson, R.A., Thies, C. and Knowles, J.D., 2018. Deep Optimisation: Solving Combinatorial Optimisation Problems using Deep Neural Networks. arXiv preprint arXiv:1811.00784.
- [2] Caldwell, J., Knowles, J., Thies, C., Kubacki, F. and Watson, R., 2022. Deep Optimisation: Transitioning the Scale of Evolutionary Search by Inducing and Searching in Deep Representations. SN Computer Science, 3, p.253.
- [3] Galván, E., 2020. Neuroevolution in Deep Neural Networks: A Comprehensive Survey. SIGEVOlution, 14(1), pp.3–6.
- [4] Galván, E. and Mooney, P., 2021. Neuroevolution in Deep Neural Networks: Current Trends and Future Challenges. IEEE Transactions on Artificial Intelligence, 2(6), pp.476–493.
- [5] Sigaud, O., 2023. Combining Evolution and Deep Reinforcement Learning for Policy Search: A Survey. ACM Transactions on Evolutionary Learning, 3(3), Article 10.





Nottingham Trent
University

Department of Computer Science

Summary

In this lecture you learnt:

- Understand neuroevolution paradigms
- Understand Deep Optimisation (DO)
- Compare gradient-based vs evolutionary methods
- Analyse hybrid Evolution + Deep RL frameworks

This lecture contributes to the following learning outcomes:

MLO2: Critically evaluate the strengths and weaknesses of different computational intelligence techniques for solving complex problems.

MLO4: Understand the ethical implications of using computational intelligence technologies and the potential impact on society.

MLO5: Apply computational intelligence techniques to solve real-world problems, demonstrating practical problem-solving skills.

MLO6: Develop proficiency in programming and using computational tools for implementing and evaluating intelligent systems.

MLO8: Communicate effectively the concepts, methodologies, and results of computational intelligence research to both technical and non-technical audiences.



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 8: Neuroevolution and Deep Optimisation for Intelligent Systems

Dr Pedro Machado pedro.machado@ntu.ac.uk